

These notes are intended to accompany the lecture slides and to provide a more continuous explanation of the main ideas. In class, many concepts are introduced briefly in order to keep the pace manageable. Here we take the opposite approach. We slow down, expand the details, and try to explain not only *what* each model is, but also *why* the field moved from one formulation to the next.

The overall story can be summarized in one sentence: *modern generative modeling is about learning how to transform a simple probability distribution into the data distribution*. Different models make different design choices about how this transformation should be represented, how it should be trained, and whether probability density can be computed explicitly.

1. GAN

Generative Adversarial Networks (GANs) approach generative modeling through a two-player game. There are two neural networks. The first is the *generator* G , which takes a simple random variable $z \sim p_z(z)$, usually Gaussian or uniform noise, and maps it to a sample $G(z)$ in data space. The second is the *discriminator* D , which receives either a real sample $x \sim p_r(x)$ or a generated sample $G(z)$ and outputs a score between 0 and 1 that is interpreted as the probability that the input is real.

The discriminator should classify real samples as real and generated samples as fake, so it tries to maximize

$$\mathbb{E}_{x \sim p_r(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))].$$

The generator, on the other hand, wants to fool the discriminator, so it tries to minimize the same objective. This leads to the minimax formulation

$$(1) \quad \min_G \max_D L(D, G) = \mathbb{E}_{x \sim p_r(x)}[\log D(x)] + \mathbb{E}_{z \sim p_z(z)}[\log(1 - D(G(z)))].$$

This formulation is elegant because it does not ask us to write down the density of the generated distribution explicitly. Instead, we only need the ability to sample from the generator. In that sense GAN is an *implicit* generative model.

1.1. Optimal discriminator

To understand what the GAN objective is really doing, we first ask: for a fixed generator G , what is the optimal discriminator?

Let p_g denote the distribution of generated samples $G(z)$. Then the loss can be rewritten as

$$(2) \quad L(D, G) = \int_x p_r(x) \log D(x) dx + \int_x p_g(x) \log(1 - D(x)) dx.$$

Notice that for each fixed location x , the discriminator value $D(x)$ appears only inside

$$f_x(\tilde{x}) = p_r(x) \log \tilde{x} + p_g(x) \log(1 - \tilde{x}),$$

where we write $\tilde{x} = D(x)$ for convenience. Since the integral sums contributions independently over all x , we can maximize pointwise. Taking derivative with respect to $\tilde{x}$,

$$(3) \quad \frac{df_x(\tilde{x})}{d\tilde{x}} = \frac{p_r(x)}{\tilde{x}} - \frac{p_g(x)}{1 - \tilde{x}}.$$

Setting the derivative to zero yields

$$\frac{p_r(x)}{\tilde{x}} = \frac{p_g(x)}{1 - \tilde{x}},$$

1

which gives

$$(4) \quad D^*(x) = \frac{p_r(x)}{p_r(x) + p_g(x)}.$$

This formula has an intuitive interpretation. If $p_r(x)$ is much larger than $p_g(x)$, then $D^*(x)$ is close to 1, meaning the discriminator is confident the sample is real. If $p_g(x)$ dominates, then $D^*(x)$ is close to 0, meaning it is likely fake. If the two distributions are equal at x , then $D^*(x) = 1/2$.

1.2. Connection to Jensen–Shannon divergence

Plugging the optimal discriminator back into the objective reveals what GAN is really minimizing. First write

$$(5) \quad L(G, D^*) = \int_x p_r(x) \log \frac{p_r(x)}{p_r(x) + p_g(x)} dx + \int_x p_g(x) \log \frac{p_g(x)}{p_r(x) + p_g(x)} dx.$$

Now recall the Jensen–Shannon divergence

$$(6) \quad \text{JS}(p_r \| p_g) = \frac{1}{2} \text{KL}\left(p_r \left\| \frac{p_r + p_g}{2}\right.\right) + \frac{1}{2} \text{KL}\left(p_g \left\| \frac{p_r + p_g}{2}\right.\right).$$

Expanding the KL terms,

$$(7) \quad \text{JS}(p_r \| p_g) = \frac{1}{2} \int_x p_r(x) \log \frac{p_r(x)}{\frac{p_r(x) + p_g(x)}{2}} dx + \frac{1}{2} \int_x p_g(x) \log \frac{p_g(x)}{\frac{p_r(x) + p_g(x)}{2}} dx$$

$$(8) \quad = \frac{1}{2} \int_x p_r(x) \log \frac{2p_r(x)}{p_r(x) + p_g(x)} dx + \frac{1}{2} \int_x p_g(x) \log \frac{2p_g(x)}{p_r(x) + p_g(x)} dx$$

$$(9) \quad = \log 2 + \frac{1}{2} L(G, D^*).$$

Therefore

$$(10) \quad L(G, D^*) = 2 \text{JS}(p_r \| p_g) - 2 \log 2.$$

So, under an optimal discriminator, training the generator is equivalent to minimizing the Jensen–Shannon divergence between the real distribution and the generated distribution. This gives a clean probabilistic interpretation of GAN.

1.3. Why GAN training is difficult

Despite the elegance of the derivation, GAN training is often unstable. One major reason is that in high-dimensional spaces the supports of p_r and p_g may lie on low-dimensional manifolds that do not overlap. In that case, there exists a discriminator that separates real and fake samples almost perfectly. When the discriminator becomes too strong, it outputs values close to 1 on real data and close to 0 on generated data. As a result, the gradient passed back to the generator becomes very small.

This creates a dilemma. If the discriminator is weak, it does not provide a meaningful training signal. If it is too strong, the gradient vanishes. This tension is one of the key motivations behind Wasserstein GAN.

2. W-GAN

Wasserstein GAN replaces the Jensen–Shannon divergence by the Wasserstein-1 distance, also called the Earth Mover’s distance. Intuitively, this distance measures the minimum amount of “work” required to transport one distribution into another.

In the continuous setting, the Wasserstein distance is defined as

$$(11) \quad W(p_r, p_g) = \inf_{\gamma \in \Pi(p_r, p_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|],$$

where $\Pi(p_r, p_g)$ is the set of all couplings between p_r and p_g , that is, all joint distributions whose marginals are p_r and p_g .

This formulation is conceptually appealing, but it is not directly practical because optimizing over all couplings is intractable. The key mathematical tool is the Kantorovich–Rubinstein duality:

$$(12) \quad W(p_r, p_g) = \frac{1}{K} \sup_{\|f\|_L \leq K} (\mathbb{E}_{x \sim p_r} [f(x)] - \mathbb{E}_{x \sim p_g} [f(x)]).$$

Here $\|f\|_L \leq K$ means that f is K -Lipschitz continuous.

If we parameterize f by a neural network f_w , then we obtain the WGAN objective

$$(13) \quad \max_{w \in W} \mathbb{E}_{x \sim p_r} [f_w(x)] - \mathbb{E}_{z \sim p_z(z)} [f_w(G_\theta(z))].$$

The role of the discriminator changes substantially here. In standard GAN, the discriminator is a classifier that outputs a probability. In WGAN, the “critic” outputs a real-valued score and is trained to approximate the dual potential in Wasserstein distance. It is no longer trying to solve a classification problem directly.

The reason WGAN is useful is that Wasserstein distance remains meaningful even when the supports of the two distributions do not overlap. This means the generator can still receive gradients that reflect how far generated samples are from the real distribution in a geometric sense.

One important practical issue is how to enforce the Lipschitz constraint. The original WGAN paper proposes weight clipping: after each gradient update, clamp each parameter into a small interval such as $[-0.01, 0.01]$. Later methods use gradient penalty or spectral normalization, which are usually more effective in practice.

3. VAE

Variational Autoencoders (VAEs) represent a very different philosophy. Instead of setting up an adversarial game, VAE starts from a probabilistic latent-variable model. We assume that data is generated by first drawing a latent variable z from a simple prior $p(z)$, usually a standard Gaussian, and then drawing x from a conditional distribution $p_\theta(x|z)$. The marginal density is

$$(14) \quad p_\theta(x) = \int p_\theta(x|z)p(z) dz.$$

If we could maximize $\log p_\theta(x)$ directly, we would have a proper likelihood-based generative model. The difficulty is that the integral is intractable for neural-network parameterizations. VAE solves this by introducing an approximate posterior $q_\phi(z|x)$ and deriving a lower bound on

the log-likelihood:

$$(15) \quad \log p_\theta(x) = \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x)]$$

$$(16) \quad = \mathbb{E}_{z \sim q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{p_\theta(z|x)} \right]$$

$$(17) \quad = \mathbb{E}_{z \sim q_\phi(z|x)} \left[\log \frac{p_\theta(x, z)}{q_\phi(z|x)} \right] + \text{KL}(q_\phi(z|x) \| p_\theta(z|x)).$$

Since the KL divergence is nonnegative, we obtain the evidence lower bound (ELBO):

$$(18) \quad \log p_\theta(x) \geq \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x, z) - \log q_\phi(z|x)].$$

Expanding $p_\theta(x, z) = p_\theta(x|z)p(z)$ gives

$$(19) \quad \log p_\theta(x) \geq \mathbb{E}_{z \sim q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)).$$

This objective has a clear interpretation. The first term is the reconstruction term: it encourages the decoder to explain the data well using the latent variable. The second term regularizes the approximate posterior toward the prior, which organizes the latent space and makes sampling possible.

VAE is attractive because it is a principled probabilistic model with tractable training. However, its samples are often blurrier than GAN or diffusion samples, partly because the reconstruction term is often implemented with Gaussian assumptions that encourage averaging.

4. DDPM: Discrete diffusion models

Diffusion models start from a simple but powerful idea: rather than learning to generate data in one shot, learn to reverse a gradual noising process.

In the discrete-time Denoising Diffusion Probabilistic Model (DDPM), we define a forward Markov chain

$$(20) \quad q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I),$$

where β_t is a small variance schedule. Repeating this process step by step corrupts the data until the distribution becomes nearly Gaussian.

A useful property is that x_t can be sampled directly from x_0 :

$$(21) \quad q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I),$$

where $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. Equivalently,

$$(22) \quad x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

The reverse process is another Markov chain

$$p_\theta(x_{t-1}|x_t),$$

whose parameters are predicted by a neural network. A standard parameterization predicts the added noise ϵ . Training then reduces to

$$(23) \quad \mathbb{E}_{x_0, \epsilon, t} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2].$$

This objective looks surprisingly simple, but it is closely connected to score matching and variational inference. One of the main reasons diffusion models work so well is that denoising at a fixed noise level is a stable supervised learning problem.

5. From DDPM to continuous time: ODE and SDE viewpoints

Once diffusion is written in continuous time, it becomes natural to describe the process by stochastic differential equations (SDEs). A general diffusion SDE has the form

$$(24) \quad dx = f(x, t) dt + g(t) dW_t,$$

where $f(x, t)$ is the drift, $g(t)$ is the diffusion strength, and W_t is standard Brownian motion.

The forward noising process can be written as an SDE. More importantly, one can derive a reverse-time SDE whose trajectories transform noise into data. In score-based generative modeling, the reverse-time SDE is

$$(25) \quad dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{W}_t,$$

where $\nabla_x \log p_t(x)$ is the score function. This equation says that if we know the score function of the intermediate distribution $p_t(x)$ at every time, then we can simulate a reverse stochastic process that generates data.

This raises an important conceptual point. Why not simply solve the reverse SDE and stop there? The answer is that while the SDE is valid, it is not always the most convenient representation. Sampling is stochastic, likelihood computation is difficult, and the presence of Brownian noise makes the dynamics harder to analyze as a deterministic transport process. This motivates the Probability Flow ODE.

6. Probability Flow ODE

A remarkable fact is that the SDE above has a deterministic counterpart, called the Probability Flow ODE:

$$(26) \quad \frac{dx}{dt} = f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log p_t(x).$$

This ODE has the *same marginal distributions* p_t as the reverse SDE, even though individual trajectories are different. This distinction is very important. The SDE and the ODE are not the same pathwise process. They do not move each individual sample along the same route. What they share is the evolution of the probability distribution.

Why is this useful? First, an ODE defines a deterministic flow. Once the initial condition is fixed, the trajectory is uniquely determined. Second, deterministic flows allow likelihood computation through the instantaneous change-of-variables formula. Third, ODE solvers often allow adaptive step sizes and efficient numerical integration.

6.1. Instantaneous change of variables

Suppose an ODE is given by

$$\frac{dx}{dt} = v(x, t).$$

Then the density transported by this flow satisfies

$$(27) \quad \frac{d}{dt} \log p_t(x_t) = -\nabla \cdot v(x_t, t),$$

where $\nabla \cdot v$ is the divergence of the velocity field. Integrating over time gives

$$(28) \quad \log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla \cdot v(x_t, t) dt.$$

This is the continuous analogue of the change-of-variables formula in normalizing flows. In discrete flows, one tracks a Jacobian determinant. In continuous flows, the log-determinant becomes an integral of divergence. This is exactly what is meant by “instantaneous change of variables.”

The key intuition is geometric. If the velocity field expands volume locally, density decreases. If it contracts volume locally, density increases. The divergence measures this local expansion or contraction rate.

7. Why move from SDEs to Probability Flow ODE?

At this point it is worth summarizing the motivation clearly, because this transition appears frequently in modern papers.

The reverse SDE is natural because it comes directly from the forward diffusion process and provides a principled probabilistic formulation. However, it has three practical drawbacks. First, trajectories are stochastic, which makes inversion and deterministic control difficult. Second, likelihood computation is not straightforward. Third, the stochastic term may require many small simulation steps.

The Probability Flow ODE resolves these issues while preserving the same marginal distributions. In this sense, the ODE is not introduced because the SDE is wrong, but because the ODE is often more convenient computationally and conceptually. It allows us to think of generation as the motion of particles under a velocity field.

8. Velocity fields

Since the ODE perspective is central to Flow Matching and Rectified Flow, we now discuss velocity fields explicitly.

A velocity field is a function

$$v : \mathbb{R}^d \times [0, 1] \rightarrow \mathbb{R}^d$$

such that the trajectory x_t satisfies

$$(29) \quad \frac{dx_t}{dt} = v(x_t, t).$$

The most important point is that a velocity field is defined on *space-time*, not on a single trajectory. For every position x and time t , it tells us the local direction in which a particle should move. Therefore, once a velocity field is known, both sample trajectories and distribution evolution are determined.

This is the key conceptual bridge to flow-based generative modeling: if we can learn a suitable velocity field that carries a simple distribution p_0 into the data distribution p_1 , then generation reduces to solving an ODE.

9. Flow Matching

Flow Matching starts from this ODE viewpoint and asks a natural question: do we really need to define an SDE and then derive the corresponding Probability Flow ODE? Or can we learn the velocity field directly?

The answer is that we can learn it directly.

9.1. Constructing training data

The training-data construction is central and is often misunderstood, so we explain it carefully. We begin by sampling

$$x_1 \sim p_{\text{data}}, \quad x_0 \sim p_0,$$

where p_0 is a simple prior such as a Gaussian. Then we sample a time

$$t \sim \mathcal{U}(0, 1).$$

Next we define a point along the straight line connecting x_0 and x_1 :

$$(30) \quad x_t = (1 - t)x_0 + tx_1.$$

This is simply linear interpolation.

For this single pair (x_0, x_1) , the entire path is a straight line and therefore moves at constant velocity. Differentiating with respect to t ,

$$(31) \quad \frac{dx_t}{dt} = x_1 - x_0.$$

So for this particular path, the target velocity is

$$(32) \quad u_t = x_1 - x_0.$$

At first sight, it may appear that the target velocity does not depend on x_t . That is true for a *single path*: once (x_0, x_1) is fixed, every point on that line has the same velocity. But the model we train is not path-specific. The model is a function $v_\theta(x, t)$ defined globally on space-time. During training, many different pairs (x_0, x_1) can produce the same or very similar x_t , but with different target directions $x_1 - x_0$. Therefore, the model is forced to average over all compatible paths.

Under mean squared error, the optimal predictor is the conditional expectation

$$(33) \quad v^*(x, t) = \mathbb{E}[x_1 - x_0 \mid x_t = x].$$

This is one of the most important formulas in Flow Matching. It explains why the method is valid. Although each individual path has constant velocity, the learned velocity field is not globally constant. Instead, it is the average direction of motion at each location and time.

The training objective is

$$(34) \quad \mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{x_0, x_1, t} [\|v_\theta(x_t, t) - (x_1 - x_0)\|^2].$$

9.2. Conditional probability paths

In the literature one often sees the statement that instead of defining the full probability path $p_t(x)$ directly, we define conditional paths $p_t(x|x_1)$. This point deserves explanation because it connects the sampling construction above to the probabilistic formulation.

If we knew the full data density analytically, we might try to define a marginal path $p_t(x)$ directly. In practice we only have data samples, not the explicit density. Flow Matching therefore works with simpler conditional paths associated with individual data samples. For a given x_1 , one defines a simple path from noise to x_1 , and then the marginal path is obtained by mixing over data:

$$(35) \quad p_t(x) = \int p_t(x|x_1) p_{\text{data}}(x_1) dx_1.$$

The important practical point is that we do not need a closed-form expression for $p_t(x)$. It is enough that we can *sample* from it. The construction

$$x_1 \sim p_{\text{data}}, \quad x_0 \sim p_0, \quad x_t = (1 - t)x_0 + tx_1$$

is exactly such a sampling procedure. So when we say that Flow Matching uses conditional probability paths, we do not mean that the algorithm explicitly computes a complicated integral. We mean that it generates training points by first sampling a data point and then constructing a simple path conditioned on that data point.

9.3. Sampling after training

Once v_θ has been learned, generation is straightforward:

$$(36) \quad \frac{dx}{dt} = v_\theta(x, t), \quad x(0) \sim p_0.$$

By integrating this ODE from $t = 0$ to $t = 1$, we obtain a sample approximately distributed according to p_{data} .

10. Why Flow Matching can still be hard

Flow Matching is conceptually simpler than score-based diffusion because it avoids learning the score function and avoids stochastic simulation. However, the regression problem can still be difficult.

The reason is variance. For a given point (x, t) , there may be many different paths passing nearby, each with a different direction $x_1 - x_0$. The conditional distribution of target velocities given (x_t, t) may therefore have large variance:

$$(37) \quad \text{Var}(x_1 - x_0 \mid x_t).$$

If this variance is large, then learning the conditional mean accurately becomes harder.

This observation leads directly to Rectified Flow.

11. Rectified Flow

Rectified Flow keeps the same basic Flow Matching idea but improves the *coupling* between source and target distributions.

11.1. Why coupling matters

In standard Flow Matching, the pairing between x_0 and x_1 is essentially random:

$$x_0 \sim p_0, \quad x_1 \sim p_{\text{data}}.$$

This means a sample from noise can be paired with any data sample. Such random pairings create many straight lines pointing in inconsistent directions. When these paths intersect or come close, the conditional variance of velocities becomes large.

Rectified Flow attempts to construct better pairings so that the induced trajectories are more coherent and easier to fit by a single velocity field.

11.2. Causalization and reflow

This terminology often causes confusion, so let us explain it carefully.

Suppose we start from some coupling between endpoints, and for each pair we define an interpolation path. These interpolation paths are not yet a causal dynamical system, because to write

$$X_t = (1 - t)X_0 + tX_1$$

we need access to both endpoints, including the future endpoint X_1 . A generative model, however, should define motion using only the current state and time. Therefore we fit a velocity field to these paths. This fitting step is sometimes called *causalization*. Formally, one minimizes

$$(38) \quad \min_v \int_0^1 \mathbb{E} [\|\dot{X}_t - v_t(X_t)\|^2] dt.$$

If the interpolation path is linear, then $\dot{X}_t = X_1 - X_0$, and this becomes exactly a Flow Matching regression objective.

This is why, in one sense, the causalization step *is* Flow Matching. The important subtlety is that the paths being fit need not always be the original random straight lines. After one round of learning, we can use the learned ODE to generate new endpoints from the same starting points. This produces a new coupling:

$$(X_0, X_1^{\text{new}}).$$

Using these updated pairs, we construct new interpolation paths and fit a new velocity field. This second stage is often called *reflow*.

Some presentations reuse the notation (X_0, X_1) after reflow, which can make it look as though the coupling has not changed. Conceptually, however, it has changed: the endpoint is no longer the original data sample paired randomly with X_0 but a new point obtained by transporting X_0 through the current flow.

11.3. Algorithmic picture

A simple way to summarize Rectified Flow is the following.

First, train an initial velocity field $v^{(0)}$ by standard Flow Matching. Then, for each source sample x_0 , solve the learned ODE

$$\frac{dx}{dt} = v^{(0)}(x, t)$$

to obtain a new endpoint $x_1^{(1)}$. This defines an updated coupling $(x_0, x_1^{(1)})$. Next, fit a new velocity field $v^{(1)}$ to the interpolation paths induced by this updated coupling. Repeat this process:

$$v^{(0)} \longrightarrow \text{reflow} \longrightarrow v^{(1)} \longrightarrow \text{reflow} \longrightarrow v^{(2)} \longrightarrow \dots$$

After k rounds, the final model is $v^{(k)}$.

11.4. Why does reflow help?

This is the most important conceptual question.

The objective itself has not changed. We are still solving a regression problem of the form

$$\mathbb{E} \|v_t(X_t) - \dot{X}_t\|^2.$$

What changes is the *data distribution* of paths. The new coupling induced by the current model tends to align trajectories better than the original random pairing. Consequently, for a given point x_t , the compatible velocities become more concentrated. In other words,

$$\text{Var}(\dot{X}_t \mid X_t = x)$$

decreases. A regression problem with lower conditional variance is easier. This is the core reason Rectified Flow improves over naive Flow Matching.

A useful informal interpretation is that reflow attempts to straighten and align the transport. Although each interpolation path remains a straight line in the training construction, the updated couplings are more compatible with the actual learned flow. This tends to reduce unnecessary crossings and contradictions between paths. In this sense Rectified Flow can be viewed as moving toward a more optimal transport coupling.

12. A final conceptual summary

It is worth pausing to connect all the models in one coherent storyline.

GAN avoids density modeling entirely and learns generation through an adversarial game. WGAN modifies the geometry of that game by replacing Jensen–Shannon divergence with Wasserstein distance. VAE introduces latent variables and learns by maximizing a tractable lower bound on likelihood. DDPM and score-based diffusion define a forward noising process and learn how to reverse it. Continuous-time SDEs provide a more general stochastic view. Probability Flow ODE shows that the same distributional evolution can be represented deterministically by a velocity field. Flow Matching then says that, once generation is viewed as solving an ODE, we may directly learn that velocity field from supervised regression. Rectified Flow observes that the quality of the learned velocity field depends heavily on the coupling used to construct training paths, and therefore proposes to iteratively improve that coupling.

The most important conceptual takeaway is this: *modern generative modeling can be understood as learning how probability mass moves*. Depending on the method, that motion may be represented adversarially, probabilistically, stochastically, or as a deterministic flow. Once this viewpoint is clear, the relationships among GAN, VAE, DDPM, ODE, SDE, Probability Flow ODE, Flow Matching, and Rectified Flow become much easier to understand.

References

- [1] Martin Arjovsky and Léon Bottou. Towards principled methods for training generative adversarial networks. *arXiv preprint arXiv:1701.04862*, 2017.
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. Wasserstein GAN. *Proceedings of ICML*, 2017.
- [3] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. *Proceedings of ICLR*, 2014.
- [4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Proceedings of NeurIPS*, 2020.
- [5] Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *Proceedings of ICLR*, 2021.
- [6] Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. *Proceedings of ICLR*, 2023.
- [7] Xingchao Liu, Chengyue Gong, and Qiang Liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. *Proceedings of ICLR*, 2023.